

Sistemas Distribuídos (DCC/UFRJ)

Módulo 4 - Parte 1 - Aula 4: Outras questões sobre a comunicação com sockets

Profa. Silvana Rossetto

- Envio e recebimento parcial de mensagens com TCP
- Envio e recebimento de dados binários
- Recomendações gerais

Envio e recebimento parcial de mensagens com TCP

Usando TCP, as funções de **envio e recebimento** (send/recv) de mensagens funcionam sobre um **buffer de rede**:

send/recv não necessariamente manipulam todos os bytes que esperamos que sejam enviados ou recebidos

Envio e recebimento parcial de mensagens com TCP

- **send** retorna quando o buffer de rede foi preenchido e **recv** retorna quando o buffer ficou vazio
- **send** e **recv** informam a quantidade de bytes manipulados
- quando **send** ou **recv** retornam 0, significa que a conexão está fechada

Cabe ao programador repetir as chamadas de send/recv até que a mensagem tenha sido completamente tratada

A repetição do **send** é **fácil de resolver** pois sabemos o tamanho da mensagem a ser enviada

Envio completo de uma mensagem via TCP

```
def meuSend(sock, msg):  
    total = 0  
    msglen = len(msg)  
    while total < msglen:  
        enviado = sock.send(msg[total:])  
        if enviado == 0:  
            #retorna com erro ou gera exceção  
        total = total + enviado
```

Função alternativa em Python: **sendall()**

Recebimento parcial da mensagem com TCP

A repetição do **recv** é **difícil de resolver** pois **NÃO** sabemos o tamanho da mensagem que foi enviada

Alternativas possíveis para garantir o recebimento completo de mensagens com TCP:

- 1 usar mensagens de tamanho fixo
- 2 usar mensagens com delimitadores
- 3 indicar o tamanho da mensagem
- 4 encerrar a conexão após o envio

!decisão a ser incorporada no protocolo de camada de aplicação!

Recebimento completo de uma mensagem via TCP

Mensagens de tamanho fixo

```
def meuRecv(sock):  
    chunks = []  
    recebidos = 0  
    while recebidos < MSGLEN:  
        chunk = sock.recv  
            (min(MSGLEN-recebidos, 2048))  
        if not chunk:  
            # retorna erro ou gera exceção  
            chunks.append(chunk)  
            recebidos = recebidos + len(chunk)  
    return b''.join(chunks)
```

Neste caso, deve-se **receber uma quantidade arbitrária de bytes e depois procurar pelo delimitador**

- **problema**: quando o emissor pode enviar uma sequência de mensagens sem esperar resposta
- uma leitura nesse caso **poderá ler parte do conteúdo da mensagem seguinte**, sendo necessário armazenar e tratar o excedente

Uma opção é usar o **primeiro byte como indicador do tipo da mensagem**, e para cada tipo ter um tamanho pre-definido

ou

usar os **N bytes iniciais para indicar o tamanho da mensagem** e fazer uma pre-leitura com repetição desses N bytes

Quando optamos por **criar uma conexão apenas para enviar uma requisição e receber a resposta**, o lado cliente sabe quando a mensagem de resposta termina ao receber 0 da função **recv**

- ex., HTTP com encerramento de conexão pelo servidor após envio da resposta

Para **transmitir dados binários entre processos remotos**, é preciso levar em conta as diferentes formas de representação desses dados

- usar funções de conversão:
- **htonl(), htons(), ntohl(), ntohs()**

- Fechar o socket para que o lado remoto não fique em uma espera indefinida (em Python, usando a sentença `with` ou chamando `close()`)
- Evitar fazer **lock** (de exclusão mútua) antes chamar uma função bloqueante
- Fazer o tratamento de todas as possíveis exceções

- 1 <https://docs.python.org/3/howto/sockets.html>
- 2 <https://realpython.com/python-sockets/>
- 3 <https://docs.python.org/3/library/socket.html>